

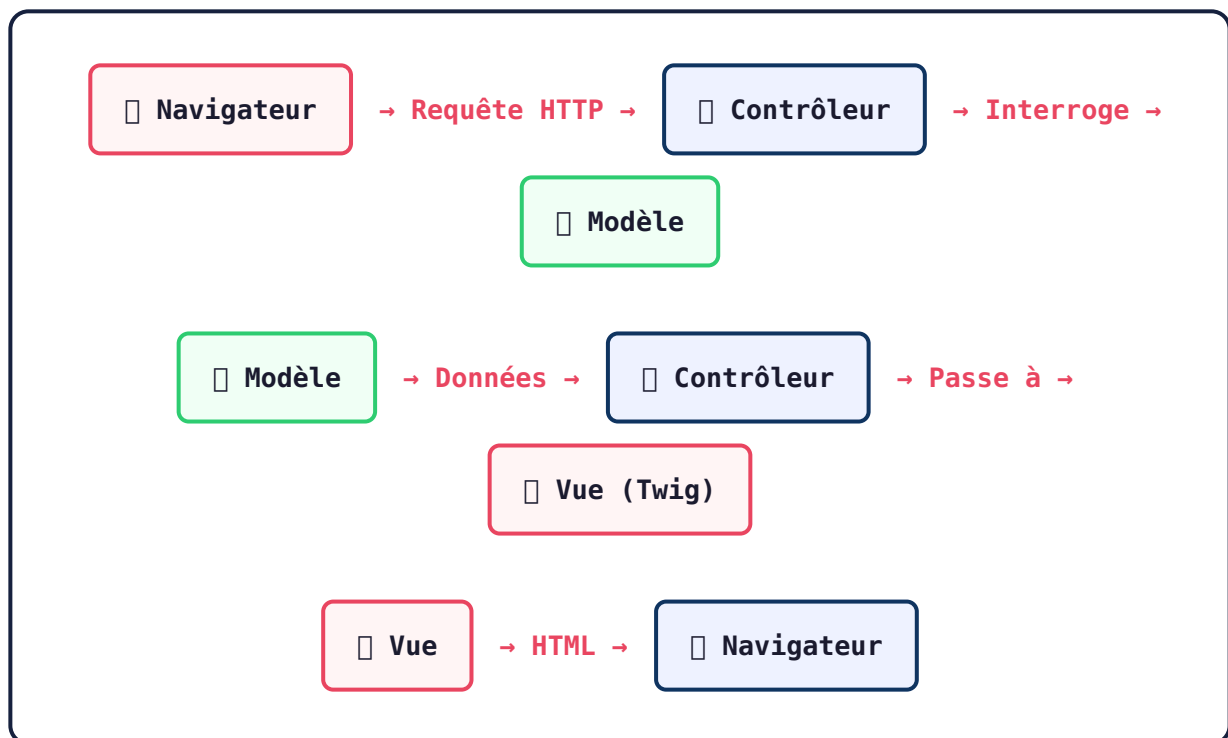
❏ Synthèse MVC, Twig, Formulaires & Sécurité

Guide pédagogique pour le projet Symfony — Garage404 | Omar

1. Le modèle MVC : Comprendre l'architecture

1.1 Qu'est-ce que MVC ?

MVC (**M**odèle - **V**ue - **C**ontrôleur) est un **patron d'architecture** qui sépare le code en 3 couches distinctes. Chaque couche a une responsabilité unique. C'est le principe **SRP** (Single Responsibility Principle).



Couche	Rôle	Dans Symfony	Exemple (ton projet)
Modèle	Gère les données et la logique métier	Entités Doctrine + Repositories	<code>src/Entity/Job.php</code> , <code>src/Repository/JobRepository.php</code>

Vue

	Affiche les données, zéro logique métier	Templates Twig (<code>.html.twig</code>)	<code>templates/jobs/jobs.html.twig</code>
Contrôleur	Reçoit la requête, orchestre Modèle+Vue	Classes avec attributs <code>#[Route]</code>	<code>src/Controller/JobsController.php</code>

□ **Règle d'or** : Un contrôleur **fin** (peu de code), un modèle **riche** (logique métier), une vue **stupide** (affichage seulement).

2. Les Contrôleurs Symfony

2.1 Anatomie d'un contrôleur

Un contrôleur est une classe PHP qui **étend** `AbstractController`. Chaque méthode publique avec un attribut `#[Route]` est une « page ».

```
// src/Controller/JobController.php
namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Routing\Attribute\Route;
use Symfony\Component\HttpFoundation\Response;
use App\Repository\JobRepository;

final class JobController extends AbstractController
{
    #[Route('/', name: 'app_jobs')]
    public function index(JobRepository $jobRepository): Response
    {
        // 1. Récupérer les données du Modèle
        $jobs = $jobRepository->findAll();

        // 2. Passer les données à la Vue
        return $this->render('jobs/index.html.twig', [
            'jobs' => $jobs,
        ]);
    }
}
```

▢ **Bonne pratique** : Le contrôleur est `final` et n'a **pas de logique métier**. Il délègue au Repository (Modèle) et passe juste les résultats à Twig (Vue).

2.2 Injection de dépendances dans les contrôleurs

Symfony **injecte automatiquement** les services dont tu as besoin dans les paramètres de méthode :

- `EntityManagerInterface $entityManager` → pour persister/supprimer
- `Request $request` → pour lire les données POST/GET
- `JobRepository $jobRepository` → pour requêter la base

2.3 Routage avec attributs

Attribut	Exemple	Signification
<code>#[Route('/jobs', name: 'app_jobs')]</code>	Route simple	GET par défaut, accessible via <code>{{ path('app_jobs') }}</code>
<code>#[Route('/jobs/{id}', name: 'app_job_show')]</code>	Route avec paramètre	<code>{id}</code> est automatiquement injecté
<code>#[Route('/jobs/new', methods: ['GET', 'POST'])]</code>	Route pour formulaire	Accepte GET (afficher) et POST (soumettre)

⚠ **Piège dans ton projet** : Le `JobsController` ne passe **aucune donnée** du modèle à la vue. Il fait juste `return $this->render(...)` avec un `'controller_name'` inutile. Le HTML est statique au lieu d'être dynamique.

2.4 Le cycle complet d'une requête

```
Navigateur → GET /jobs → #[Route('/jobs')]
      ↓
      JobController::index()
      ↓
      $jobRepository->findAll() ← MODÈLE
      ↓
      $this->render('jobs/index.html.twig', ['jobs' => $jobs])
      ↓
```

Twig compile jobs/index.html.twig → HTML ← *VUE*

↓

Response HTML envoyée au navigateur

3. Les Vues avec Twig

3.1 Le principe d'héritage (CRUCIAL !)

Twig permet de définir un **template parent** (le layout) et des templates enfants qui ne redéfinissent que ce qui change. C'est l'**anti-duplication**.

Le template parent : `base.html.twig`

```
{# templates/base.html.twig #}
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>{% block title 'Bienvenue' %}</title>
  <link rel="stylesheet" href="/css/app.css">
  {% block stylesheets %}{% endblock %}
</head>
<body>
  {% include '_partials/_header.html.twig' %}

  {% block body %}{% endblock %}

  {% include '_partials/_footer.html.twig' %}
  {% block javascripts %}{% endblock %}
</body>
</html>
```

Un template enfant : `jobs/index.html.twig`

```
{% extends 'base.html.twig' %}

{% block title %}Offres d'emploi – Luxury Services{% endblock %}

{% block body %}
  <h1>Nos offres d'emploi</h1>

  {% for job in jobs %}
    <article>
      <h2>{{ job.jobTitle }}</h2>
      <p>{{ job.description }}</p>
      <a href="{{ path('app_job_show', {id: job.id}) }}">Voir l'offre</a>
    </article>
  {% else %}
    <p>Aucune offre pour le moment.</p>
  {% endfor %}
{% endblock %}
```

```
{% endfor %}
{% endblock %}
```

⚠ **Ce qui ne va PAS dans ton projet :** Chaque template (home, jobs, contact, compagny) répète intégralement le `<head>`, les 30 imports JS/CSS, et la navigation. Utilise l'héritage Twig (`{% extends 'base.html.twig' %}`) à la place !

3.2 Les concepts clés Twig

Syntaxe	Usage
<code>{{ variable }}</code>	Afficher une variable
<code>{% if condition %}...{% endif %}</code>	Condition
<code>{% for item in collection %}...{% endfor %}</code>	Boucle
<code>{% block nom %}...{% endblock %}</code>	Zone remplaçable par l'enfant
<code>{% extends '...' %}</code>	Hériter d'un template parent
<code>{% include '...' %}</code>	Insérer un partial (header, footer)
<code>{{ path('route_name', {id: val}) }}</code>	Générer une URL à partir du nom de route
<code>{{ form_start(form) }}</code>	Ouvrir un formulaire Symfony

3.3 Partial : factoriser les éléments répétitifs

Ton header et ton footer doivent être dans des partials :

```
templates/
├─ base.html.twig           ← Le layout parent
├─ _partials/
│   ├─ _header.html.twig    ← Navigation + CSS/JS communs
│   └─ _footer.html.twig    ← Footer commun
├─ home/
│   └─ index.html.twig      ← {% extends 'base.html.twig' %}
├─ jobs/
│   ├─ index.html.twig
│   └─ show.html.twig
```

```
└─ contact/  
  └─ contact.html.twig
```

4. Les Formulaires Symfony

4.1 Pourquoi utiliser le composant Form ?

- □ Génération automatique du HTML du formulaire
- □ Validation automatique côté serveur
- □ Protection CSRF intégrée (token automatique)
- □ Mapping automatique Entité ↔ Formulaire

4.2 Créer un FormType

```
// src/Form/JobType.php
namespace App\Form;

use App\Entity\Job;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;

class JobType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            // Chaque ->add() correspond à un champ de l'entité
            ->add('jobTitle') // Champ texte simple
            ->add('jobCategory')
            ->add('jobType', ChoiceType::class, [ // Champ avec options
                'choices' => [
                    'CDI' => 'cdi',
                    'CDD' => 'cdd',
                    'Freelance' => 'freelance',
                ],
            ])
            ->add('description', TextareaType::class) // Textarea
            ->add('active', CheckboxType::class, [
                'required' => false,
            ])
        ;
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => Job::class, // Lie le formulaire à l'entité Job
        ]);
    }
}
```



```
}  
}
```

4.3 Utiliser le formulaire dans le contrôleur

```
use App\Entity\Job;  
use App\Form\JobType;  
  
#[Route('/job/new', name: 'app_job_new', methods: ['GET', 'POST'])]  
public function new(Request $request, EntityManagerInterface $em): Response  
{  
    // 1. Créer l'entité  
    $job = new Job();  
  
    // 2. Créer le formulaire lié à l'entité  
    $form = $this->createForm(JobType::class, $job);  
  
    // 3. Traiter la requête (remplit $job si POST)  
    $form->handleRequest($request);  
  
    // 4. Si soumis ET valide → sauvegarder  
    if ($form->isSubmitted() && $form->isValid()) {  
        $em->persist($job);  
        $em->flush();  
  
        return $this->redirectToRoute('app_jobs');  
    }  
  
    // 5. Afficher le formulaire (GET ou erreurs)  
    return $this->render('job/new.html.twig', [  
        'form' => $form,  
    ]);  
}
```

4.4 Afficher le formulaire dans Twig

```
{% extends 'base.html.twig' %}  
  
{% block body %}  
    <h1>Créer une offre d'emploi</h1>  
  
    {{ form_start(form) }}  
        {{ form_row(form.jobTitle) }}  
        {{ form_row(form.jobCategory) }}  
        {{ form_row(form.jobType) }}  
        {{ form_row(form.description) }}  
        {{ form_row(form.active) }}  
  
    <button type="submit" class="btn">Publier l'offre</button>
```

```
{{ form_end(form) }}  
{% endblock %}
```

▢ **Astuce :** `form_row()` affiche le label + le champ + les erreurs automatiquement. Pour plus de contrôle, utilise `form_label()`, `form_widget()` et `form_errors()` séparément.

5. La Sécurité dans Symfony

5.1 Le hashage des mots de passe (CRITIQUE)

❑ **Ne JAMAIS stocker un mot de passe en clair.** Symfony fournit `UserPasswordHasherInterface` pour hasher automatiquement avec bcrypt/argon2.

Dans le contrôleur d'inscription :

```
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;

public function register(
    Request $request,
    UserPasswordHasherInterface $passwordHasher, // ← Injecté
    EntityManagerInterface $em
): Response {
    $user = new User();
    // ... formulaire ...

    if ($form->isSubmitted() && $form->isValid()) {
        // ❑ Hasher le mot de passe AVANT de persister
        $hashedPassword = $passwordHasher->hashPassword(
            $user,
            $form->get('plainPassword')->getData()
        );
        $user->setPassword($hashedPassword);

        $em->persist($user);
        $em->flush();
    }
}
```

⚠ **Dans ton projet :** L'entité `Candidature` a un champ `password` stocké EN CLAIR (pas de hashage). C'est une faille de sécurité majeure. Le `RegistrationController` lui, le fait correctement pour `User` → applique la même logique pour `Candidature`.

5.2 La protection CSRF

Le **Cross-Site Request Forgery** est une attaque qui force un utilisateur à exécuter des actions non désirées. Symfony protège automatiquement les formulaires avec un token CSRF.

```
{# Dans un formulaire Symfony, le token CSRF est automatique #}  
{{ form_start(form) }} ← Inclut un champ caché _token  
{{ form_end(form) }} ← Ferme le formulaire
```

Pour les formulaires **non-Symfony** (HTML pur), utilise :

```
<form method="post">  
    <input type="hidden" name="_token"  
        value="{{ csrf_token('delete_job') }}">  
</form>  
  
// Dans le contrôleur :  
if ($this->isCsrfTokenValid('delete_job', $request->getPayload()->getString('_token'))) {  
    // Action autorisée  
}
```

5.3 Les variables d'environnement (.env)

❗ Le fichier **.env** ne doit JAMAIS être commité sur GitHub.

Fichier	Commité ?	Contient
<code>.env</code>	❗ NON	Secrets réels de DEV
<code>.env.example</code>	❗ OUI	Structure sans valeurs réelles
<code>.env.local</code>	❗ NON	Surcouche locale jamais commitée
<code>.env.prod</code>	❗ NON	Secrets production

5.4 Configuration de la sécurité

```
# config/packages/security.yaml  
security:  
    password_hashers:  
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
```

```
providers:
  app_user_provider:
    entity:
      class: App\Entity\User
      property: email

firewalls:
  main:
    lazy: true
    provider: app_user_provider

access_control:
  # Protéger les routes admin
  - { path: ^/admin, roles: ROLE_ADMIN }
  - { path: ^/profile, roles: ROLE_USER }
```

6. Checklist Pratique pour ton Projet Luxury-Service

#	Action corrective	Priorité
1	Hasher tous les mots de passe (<code>Candidature.password</code>)	☐ CRITIQUE
2	Ajouter <code>.env</code> au <code>.gitignore</code> , créer <code>.env.example</code>	☐ CRITIQUE
3	Refactoriser TOUS les templates avec <code>{% extends 'base.html.twig' %}</code>	☐ HAUTE
4	Supprimer <code>public/templates/</code> (doublons HTML statiques)	☐ HAUTE
5	Remplacer les liens en dur par <code>{{ path() }}</code> dans tous les templates	☐ HAUTE
6	Connecter les contrôleurs à la BDD (passer des données réelles aux vues)	☐ MOYENNE
7	Nettoyer les fichiers <code>.old</code> , <code>.gitkeep</code> inutiles	☐ MOYENNE
8	Ajouter des <code>access_control</code> dans <code>security.yaml</code>	☐ MOYENNE
9	Ajouter des contraintes de validation (<code>#[Assert\NotBlank]</code> , etc.) aux entités	☐ BASSE
10	Créer des méthodes personnalisées dans les Repositories	☐ BASSE

7. Résumé : Les 7 commandements MVC+Sécurité

1. **Contrôleur fin** : Il orchestre, il ne fait pas de logique métier
2. **Modèle riche** : La logique métier est dans les entités et repositories
3. **Vue héritée** : Toujours utiliser `{% extends 'base.html.twig' %}`
4. **Formulaire typé** : Toujours utiliser un FormType lié à une entité

5. **Password hashé** : JAMAIS de mot de passe en clair en base
6. **Routes nommées** : Toujours utiliser `{{ path() }}` , jamais de lien en dur
7. **.env ignoré** : Le fichier `.env` n'est jamais commité